

32 张 NPU、6/7 个 SSU：公式、策略与同输入实验

固定中间一秒 + 完整请求集；收益、失败与信息边界一起报告

2026-09-06

Contents

1 先给结论：不能直接照搬四卡结果	1
2 到底给了系统什么输入？	2
2.1 单位与拓扑	2
2.2 请求参数：不随意指定计算时间	2
2.3 平均总量不够，还要看每一盘	2
3 比较的是哪些策略？	3
4 同一秒与完整请求集必须一起看	3
4.1 不移动主窗口，但检查窗口敏感性	4
4.2 原配比不减量；过载组不隐藏	5
4.3 总量接近容量，但最热盘已经超载	5
4.4 种子 06 的近容量消融及新候选复核	5
5 数学和 Python 到底能匹配到什么程度？	6
5.1 为什么层间自然错开不是保证？	6
6 控制成本和现实边界	7
7 文件和复现	7

1 先给结论：不能直接照搬四卡结果

本报告是实际运行原项目仿真器的结果，不是把四卡利用率按比例外推。预测数学已扩展到每盘 FIFO、每卡唯一接收链路和层间反馈；性能实验则分别比较原 Baseline、原 Once per layer、新 A 和新 B。

必须分开三个问题：公式在限定模型中是否正确、信息不足时在线预测是否可靠、调度是否大幅提高利用率。前者成功不自动证明后两者。

下表为热点不超盘容量的变化输入。所有数字都是 NPU 的计算占比（%），不是 SSD 忙碌率。06/07 是输入排列种子末两位；同一行严格共用外部请求、到达时间、SSD 放置与原生提交种子。

注意：为分别满足每盘容量，6 盘和 7 盘 near 配额不同；本表用于同一行的策略比较，**不能把两行差值全部归因于多加一盘**。第 4 节的原配比过载组才保留相同用户请求参数与到达规律，仅按各自 SSU 数量重新执行原生数据放置。

SSU	种子	Baseline	Once	A	B	A-Once
6	06	93.260	94.832	95.731	95.216	+0.899
6	07	98.726	99.384	99.731	98.656	+0.347
7	06	97.360	98.637	99.678	99.046	+1.041
7	07	98.591	99.443	99.695	99.262	+0.251

A 固定原 NPU，只调整独占 Path 的 CIR；B 在同一 A 上增加到达时选卡。A-Once 是百分点，不是相对速度百分比。表中负值必须保留，不能只挑一组赢家。

本轮所有已保存性能实验共 44 组；固定窗口内每张卡始终有已接纳工作：全部通过。本轮最高窗口值为 99.732% (6 盘, 20260907, 局部 stall 交换), 但它不构成最优证明, 也不等于所有种子、P99 和整批完成时间都最好。**近容量原始 data 变化输入没有自动重现旧构造中“baseline 很低、新策略几乎 100%”的大幅差距。**

2 到底给了系统什么输入？

2.1 单位与拓扑

- 32 张 NPU 独立处理不同请求，不是同一请求做 32 卡张量并行。
- 每 SSU 一个非抢占后端，40 GiB/s；每 NPU 一个 50 GiB/s FCFS 接收链路，所有 SSU 的返回共同排队。
- 一条 I/O 就是一块：128 token 对应的 GLM 每层 KV, 176 KiB = 180224 byte。这里不用十进制 GB/s。
- 每请求 8 层，batch=1，启用跨请求 layer0 预取。只有前请求最后一层开始计算时已到达的 successor 才能被预取；不补造迟到预取。
- 数据载荷 SSD→HBM，无 DRAM 缓存/Prefill 命中收益。控制元数据不是把 KV 载荷放入 DRAM。

2.2 请求参数：不随意指定计算时间

直接解析项目 data，验证原文件 SHA256；使用其中计算时长和读取量，未插值、未更换计算模型。下表是每个原始 NPU 的 22 请求配额（种子 06），随后每卡独立打乱顺序。长度 K 表示乘 1024 token；NQL 是本次查询 token 数。

长度 K	NQL	块/层	MiB/层	C/ms	6 盘配额	7 盘配额
32	128	255	43.828	1.178026	4	4
32	256	254	43.656	1.997480	2	2
64	512	508	87.312	6.386487	2	2
96	512	764	131.312	9.070842	2	2
192	512	1532	263.312	17.123906	4	4
192	1024	1528	262.625	34.100782	3	5
192	2048	1520	261.250	68.056186	5	3

每层读取块数 $K = (\text{total tokens} - NQL)/128$ ；每层数据量 $V = K \times 180224/2^{30}$ GiB。C 是 data 给出的单层计算毫秒数； $V/(C/1000)$ 才是无 stall 计算节奏下的名义 GiB/s，不是“读取所需时间”。物理服务时间须除以设备实际服务带宽。

data 是参数表，不是线上概率分布或到达 trace。表中取值是真实项目输入，配额、随机顺序与到达规律是本实验构造，不能声称这是线上出现频率。

每卡最初恰有 4 请求在 $t = 0$ 到达；其余按该卡理想计算节奏提前三个请求间隔到达。令请求 0 起编号、 C_j 为单层计算时长：

$$a_j = \max \left(0, \sum_{k < j} 8C_k - \sum_{k < 3} 8C_k \right).$$

每组总计 704 请求。初始 backlog 是额外已到达工作，因此本文的“名义需求”不冒充整段 trace 的入口 offered bandwidth。所有策略保留相同到达，B 只能改变到达时的卡分配。

2.3 平均总量不够，还要看每一盘

对原固定分配，统计每卡完整输入总计算 T_i (ms) 和逐盘读量 V_{is} (GiB)：

$$d_{is} = 1000V_{is}/T_i, \quad d_s = \sum_i d_{is}.$$

这按计算时间加权，不是直接平均每个请求的 V/C 。若所有卡要按原完整配比无 stall 持续计算，至少需要每个 $d_s \leq 40$ 、每卡 $\sum_s d_{is} \leq 50$ 。这不是任意有限一秒利用率的上界，也不是 B 重分配后的实时需求事实。

原生一致性哈希保留不动；6 盘 near 总 218.316 GiB/s、最热 39.877；7 盘 near 总 249.203、最热 39.159。因此 near 是“热点接近容量”，总容量占比约 90.96% 和 89.00%，不是统一 98%。原始配比直接扩到 32 卡则为 316.183，6/7 盘都承受不了。

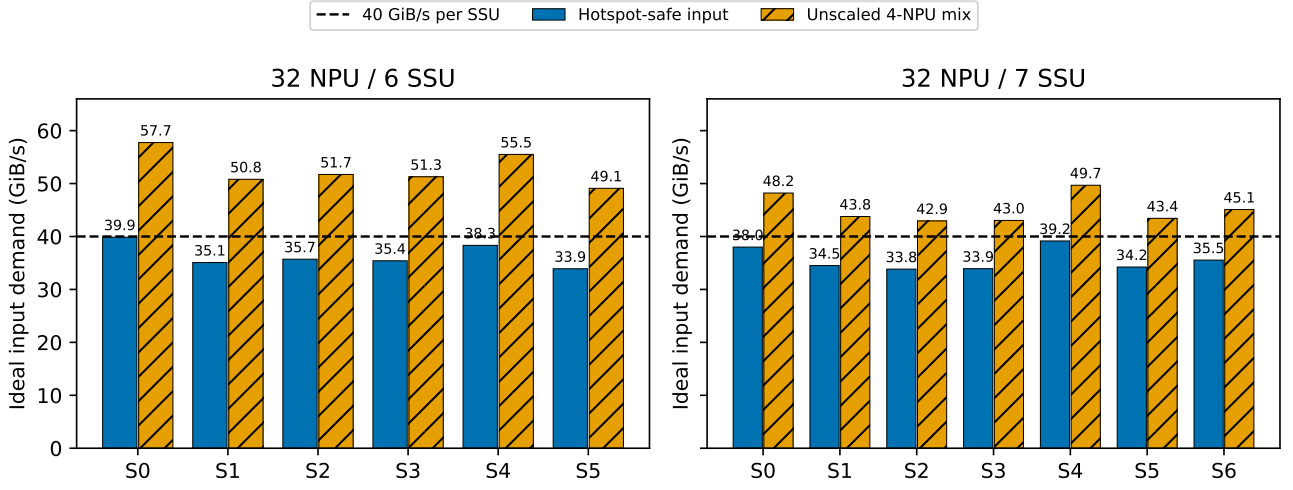


Figure 1: 每根柱为完整输入的计算时间加权名义需求，不是某一秒实际盘吞吐。虚线为单盘物理容量；橙色直接放大旧配比，蓝色调整配额使每盘不过载。

3 比较的是哪些策略？

Baseline 在每一 SSU 都只用 Path0 FIFO。Once/layer 保留项目原 layer_once 实现；它也会选择 Path，不能把 A 超过 Once 简单归因于“拥有多个 Path”。

A 使用每 NPU 一条独占 Path，在每 SSU 的 256 条中实际只占 32 条；ID 由原 dedicated_path_id 生成。在线取得已释放层的逐盘未完成量 w_{is} 和数据 deadline D_i ，按最早 deadline 排序；给同一层的各盘分量按同一个进度比例分配，使：

$$r_{is} = p_i w_{is}, \quad \sum_i r_{is} \leq 40, \quad \sum_s r_{is} \leq 50.$$

一个层必须等所有盘的数据到齐，不能只优先读完最容易的一盘。已经在计算时，下层 deadline 是**当前层计算结束时刻**；跨请求 L0 也用前驱末层结束时刻，不用新请求自己的计算时间。已 stall 的层已经错过 deadline，需要尽快启动，但不同 overdue 层之间也存在选择冲突。

这是 CIR 优先方向，不是替换 SSD 的真实 EDF：原来的虚拟完成标签、非抢占命令、借带宽机制仍保留。项目不支持有限 PIR，本轮使用 PIR= 无穷。**CIR 行和 ≤ 50 不意味着实际同时从多盘返回的峰值 ≤ 50** ；接收链路仍然会排队。

B 在每次真实到达时，枚举 32 个卡位置，按已知剩余计算及每盘/每卡工作量的流体完工评分选卡。不读取未来请求，也不迁移已分配请求，不改变其数据所在 SSU。这是后发式，不是全局最优或精确 deadline 预测器。

另做三类消融：V/C 共同进度 max-min、按剩余时间预留再分配、只按剩余计算选卡；以及 0.1 ms 控制最小间隔。收益须从结果看，不能因公式看起来合理就预设胜出。

最后增加一个有针对性的 stall_interchange 候选：先按 deadline 排列，做两遍有限相邻交换。把每层独占服务下界作为串行代理服务时间 T_i ，当前还能计算的时间为 $d_i = \max(0, D_i - \text{now})$ ，前缀累计时间为 q 。比较：

$$L_{ij} = (q + T_i - d_i)^+ + (q + T_i + T_j - d_j)^+.$$

仅当反序的 L_{ji} 更小时交换。共享单瓶颈时，例如大任务需 10ms 且已到期，小任务需 1ms 且 2ms 后到期，总新增等待由 19ms 变 11ms；多 SSU 并行下该串行代理不是精确预测。两遍共至多 $2(N-1)$ 次比较，不穷举，可能偏向短任务，必须检查 P99。

4 同一秒与完整请求集必须一起看

主指标固定为所有策略同一绝对区间 [1000, 2000] ms：

$$U = \frac{\sum_{i=0}^{31} \text{compute_overlap}_i}{32 \times 1000 \text{ ms}}.$$

每个 batch 接纳至完成的时间与窗口求交，检查每卡 active=1000 ms；然后将 active-compute 标为 I/O 等待，1000-active 标为无已接纳工作 idle。这避免把入口空闲伪装成 I/O stall，也没有移动窗口挑最大提升。

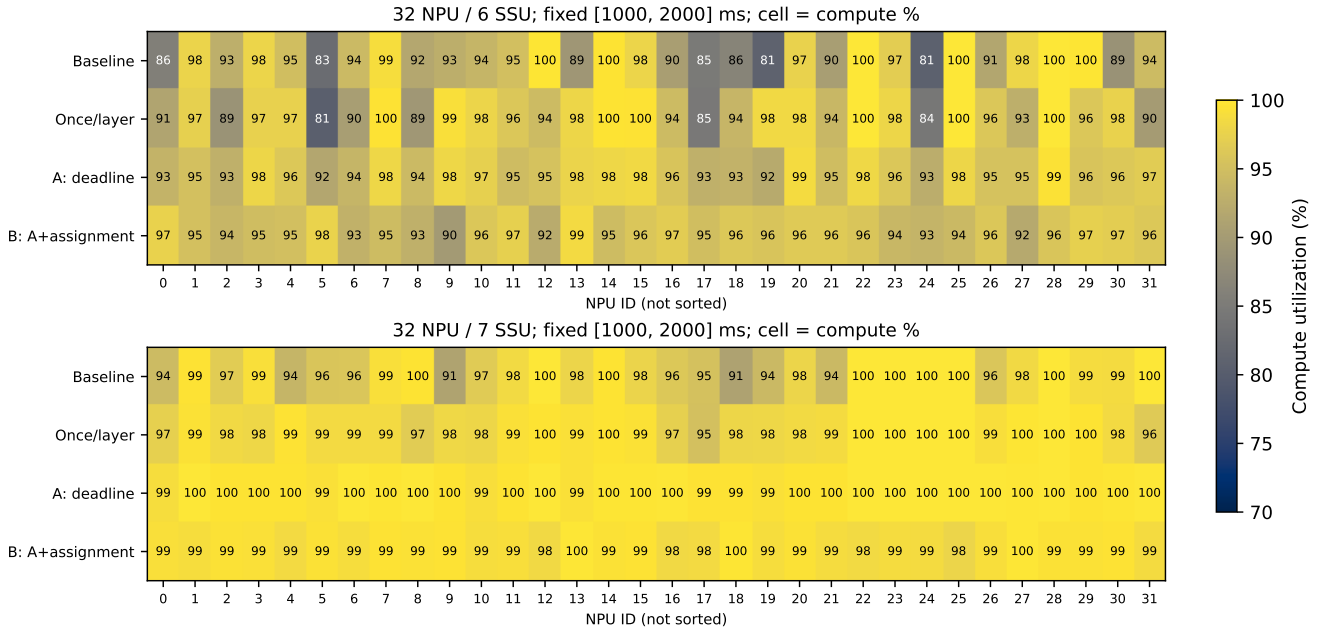


Figure 2: 格中整数为该卡计算利用率（%），横轴是真实卡号，没有逐策略重新排序；色阶固定 70-100%。精确小数保存于 JSON。这里只画种子 06，同一秒比较。

下表覆盖完整 704 请求，不是窗口内标记的一部分。平均/P99 延迟从外部到达算至完成，包含 admission 排队。整批利用率以 32 卡 × 完整 makespan 为分母，包含启动与尾部空闲。

SSU	种子	策略	完工 ms	均延迟 ms	P99 ms	整批 U%
6	06	Baseline	4808.4	986.2	2000.8	91.62
6	06	Once/layer	4731.6	952.2	1970.5	93.11
6	06	A: deadline	4643.9	917.9	1875.9	94.87
6	06	B: A+fluid 选卡	4930.7	889.3	1365.4	89.35
6	07	Baseline	4882.6	951.0	1936.0	90.23
6	07	Once/layer	4767.5	928.7	1960.0	92.41
6	07	A: deadline	4694.0	901.8	1969.2	93.86
6	07	B: A+fluid 选卡	4836.4	849.4	1318.7	91.09
7	06	Baseline	4111.6	840.9	1679.3	93.94
7	06	Once/layer	4032.9	819.0	1687.5	95.77
7	06	A: deadline	3946.5	783.1	1669.4	97.87
7	06	B: A+fluid 选卡	4234.1	758.8	1223.8	91.22
7	07	Baseline	4247.2	789.1	1536.7	90.94
7	07	Once/layer	4184.5	769.7	1574.1	92.30
7	07	A: deadline	4071.2	756.3	1578.3	94.87
7	07	B: A+fluid 选卡	4222.7	705.4	1136.3	91.47

选卡可能降低多数请求或 P99 延迟，却使尾部某些 NPU 更晚结束。同时看 makespan 与一秒利用率，才能发现这种目标冲突；不能把中间一秒变好一律解释为吞吐变好。

4.1 不移动主窗口，但检查窗口敏感性

下面额外检查相同请求在 [2000, 3000] ms 的计算占比，不替换用户要求的主窗口。各策略经历请求画像、相位不同，一秒读数不能直接当作长期稳态。

SSU	种子	Baseline%	Once%	A%	B%	全卡 active
6	06	98.643	99.461	99.649	99.736	是
6	07	96.670	97.626	99.468	95.307	是

SSU	种子	Baseline%	Once%	A%	B%	全卡 active
7	06	98.209	99.003	99.612	99.617	是
7	07	98.765	99.292	99.606	99.661	是

4.2 原配比不减量：过载组不隐藏

SSU	需求	总容量	最热盘	Baseline%	Once%	A%	B%
6	316.18	240	57.75	66.779	67.538	68.216	71.819
7	316.18	280	49.68	79.083	78.814	78.843	83.763

这里低利用率包含物理容量不足，不能要求只靠 Path/CIR 把所有卡长期推到 100%。调度可以改变受害者、相位及有限窗口中的计算组合，却不能让盘多读出字节。某一秒的平均计算占比也不能直接套“总容量/全输入平均需求”作为硬上界。

4.3 总量接近容量，但最热盘已经超载

SSU	总需求	最热盘	Baseline%	Once%	A%	B%
6	232.744	42.508	91.891	93.030	94.302	98.394
7	268.151	42.135	95.407	95.637	97.249	96.989

该组的总量约为总容量的 96%-97%，但最热盘仍超过 40 GiB/s。它直接检验“只看所有 SSU 加总”会遗漏什么；不把它列为每盘都满足的 near 组。B 改变卡位置不改变已有数据的落盘位置。

特别看 6 盘：B 把主窗口利用率从 91.891% 提高到 98.394%，但完整请求集的完工时间仅从 4658.8ms 变为 4643.5ms，缩短 0.33%；仍慢于 A 的 4458.4ms。不能把这一秒的约 6.5 个百分点增益当作同幅度的长期吞吐提升。

4.4 种子 06 的近容量消融及新候选复核

SSU	策略	窗口 U%	完工 ms	P99 ms
6	Baseline	93.260	4808.4	2000.8
6	A+compute 选卡	96.379	4932.3	1361.1
6	B: A+fluid 选卡	95.216	4930.7	1365.4
6	A: deadline	95.731	4643.9	1875.9
6	A: deadline / 0.1ms 限频	95.770	4641.9	1871.8
6	deadline 预留	95.698	4634.8	1882.2
6	V/C max-min	94.131	4769.6	1983.6
6	Once/layer	94.832	4731.6	1970.5
6	局部 stall 交换	96.068	4616.9	1891.3
7	Baseline	97.360	4111.6	1679.3
7	A+compute 选卡	98.615	4293.8	1242.2
7	B: A+fluid 选卡	99.046	4234.1	1223.8
7	A: deadline	99.678	3946.5	1669.4
7	A: deadline / 0.1ms 限频	99.676	3948.2	1668.8
7	deadline 预留	99.633	3945.2	1669.5
7	V/C max-min	97.979	4035.6	1688.9
7	Once/layer	98.637	4032.9	1687.5
7	局部 stall 交换	99.673	3954.0	1667.4

局部 stall 交换按两个种子单独对照 A；下面时间差都是“新候选减 A”，完工或 P99 的正值表示变差。它不是每个指标都优于 A，也不能因单瓶颈代理目标下降就宣布真实多盘目标单调下降。

SSU	种子	A%	交换%	差/百分点	完工差 ms	P99 差 ms
6	06	95.731	96.068	+0.3372	-27.0	+15.4
6	07	99.731	99.732	+0.0003	+8.6	-17.2
7	06	99.678	99.673	-0.0049	+7.5	-2.1
7	07	99.695	99.703	+0.0081	+10.9	+0.0

这些是受控策略差异的证据，不是每条命令的唯一因果归因。Deadline、长计算的隐藏能力、数据分盘偏斜、CIR 虚拟历史、接收链路、到达与请求切换相位都可能共同影响结果。

5 数学和 Python 到底能匹配到什么程度？

配套 multi_ssu_math.pdf 展开所有公式与输入含义；multi_ssu_stall_predictor.py 不调用原仿真器，独立实现标准库闭环递推。

- **严格筛查**：全部未提交的目标层，可用每盘前方工作下界和接收链路下界证明 must_stall；未超过 deadline 只返回 unknown，不误称必定满足。
- **已经部分提交的当前层**：screen_pending_layer 使用自己的 queued/unissued/link 数量给严格下界，不能把整条 Path 的全部其他 I/O 都当作自己前方工作。全部数据已到 HBM 时返回 already_ready，不凭此判断过去是否迟到。512 次激活层和 192 次部分提交快照检查均无下界越界。
- **有完整冷启动输入的条件预测**：本轮 10 个 32 卡 6/7 盘 case、共 3088 层，对原生到齐、计算开始、未来 stall 的最大绝对误差 0 ms。包含原 data 多种请求和单卡接收瓶颈回归。相同初始输入/原生公开种子允许重现同刻提交顺序，不是读取结果后重放。
- **运行中只有数量的预测**：不知道 SSU FIFO 的归属顺序、已经服务多少以及同刻排列时，必须重建一种假设。验证包含非空队列、正在计算的卡和新请求到达；存在毫秒级误差和漏报。保存的场景采样范围不是数学上下界。

本轮真实 warm 验证详情直接保存在 data/validation_multi_ssu_predictor.json；不能将冷启动的 0 误差泛化到线上计数预测。新 A 使用实际可观测 deadline/剩余量做 CIR 决策，**没有声称完整 baseline 预测器能精确模拟 256 Path CIR 调度**。

warm 验证不是只输入 Path 总数量：还使用按请求/层/SSU 维护的 SSD 完成计数、客户端发射轮转状态和 NPU 链路剩余量。SSD 完成计数需要 SSU 回报或实验 instrumentation；此时仍不知道盘内顺序。A/B 在线策略只用 HBM 完成回执的保守剩余量，两者预测条件不同。

SSU	快照 ms	正在计算	到齐误差	stall 误差	漏报	误报
6	0.002	0	0.303724	0.252209	0	0
6	0.04	0	0.535275	0.269394	1	0
6	2.0	24	1.020215	1.020215	0	0
7	0.002	0	0.233628	0.188928	0	0
7	0.04	0	0.462701	0.239082	1	0
7	2.0	15	1.261261	0.889694	0	1

表中误差单位为 ms，漏报/误报仅统计非 L0 的 stall；“正在计算”列确认测试不只是初始空系统。该样本共 2 次漏报、1 次误报，不是经过统计认证的错误概率。32 卡 raw-data 八层完整预测约 2.5 秒，故完整递推用于离线分析，不能每层实时调用。

5.1 为什么层间自然错开不是保证？

“发起时间不同”不等于“每次读完都赶得上”。一层的到齐受最慢 SSU 和同卡接收队列约束。只要某个必经盘前方不可绕过的剩余工作 H_s 足够长，即使卡改变层相位，也可能赶不上：

定义一条 I/O 的毫秒服务时间 $\tau_s = 1000b/(2^{30}B_s)$ 、 $\ell_i = 1000b/(2^{30}L_i)$ ，其中 $b = 180224$ byte， B_s, L_i 用 GiB/s。条件为：

$$H_s + K_{is}\tau_s + \ell_i > 2C_i.$$

在一层前瞻、受害者正计算、还有至少两个后继转移且画像不变时，这是接下来两个转移中至少一次 stall 的充分条件；左侧减 $2C_i$ 是**两次累计 stall**下界，不是单层 stall 下界。它不成立不代表没 stall。有限 8 层也不能凭一次证书宣称无限循环必然持续。

另一个不可自然错开的因素是单卡接收容量。原 data 的 192K/NQL128 有 1535 块, $C \approx 4.533469$ ms; 六/七盘足够快地汇聚时, 单卡仍至少需 $\tau + 1535\ell \approx 5.157089$ ms 才到齐。单活跃卡原生实测后续每层等待约 0.623620 ms。这里没有其他卡可供错开; 这是机制回归, 不是 32 卡利用率案例。

6 控制成本和现实边界

SSU	策略	重算次数	Path 写次数	均墙钟/us	总墙钟/s
6	Baseline	0	0	0.0	0.00
6	A: deadline	40105	399336	87.3	3.50
6	A: deadline / 0.1ms 限频	25504	287835	192.8	4.92
6	Once/layer	0	0	0.0	0.00
7	Baseline	0	0	0.0	0.00
7	A: deadline	45722	681842	149.6	6.84
7	A: deadline / 0.1ms 限频	25357	426231	113.7	2.88
7	Once/layer	0	0	0.0	0.00

计数覆盖完整请求集; planner 是远程 Python callback 单次/累计实测墙钟耗时, 不是严格 CPU 时间, 受并行负载影响, 不是硬件固有延迟。它还**不包括**原生控制快照、pressure 收集、CIR 应用等开销。零值的 Once 不代表没有客户端路径规划成本, 只表示没有使用新 CIR 控制器。

在线选卡另有成本, 下表是种子 06 的 704 次到达决策, 只计选卡 hook, 不能与 CIR callback 漏项互相替代:

SSU	策略	决策数	均墙钟/us	总墙钟/s
6	A+compute 选卡	704	1368.9	0.964
6	B: A+fluid 选卡	704	1130.6	0.796
7	A+compute 选卡	704	1432.5	1.008
7	B: A+fluid 选卡	704	1916.1	1.349

A 默认在实际层开始、层到齐、每盘分量到 HBM 完成时更新, 后者最多每层每盘一次; 32 卡 $\times 6/7$ 盘的事件数会明显超过 Once。SSU 分量完成回执可释放失效预算, 但更频繁的全局协调有成本。0.1 ms 限频只是限制更新时间, 不是完整控制通信延迟仿真。

仿真没有把控制器计算、跨卡通知或 CIR 寄存器写入延迟加到数据面时间。因此表中的提升不能直接兑现为硬件收益。实际部署还要量化遥测年龄、时钟同步、控制接口吞吐、计算时长预测误差和控制失败。SSD 介质内部并行、GC、读放大与链路拓扑也不是本单后端模型完整覆盖的现实。

A/B 在线策略不调用 DRAM 缓存、不取得隐藏 FIFO; 客户端未完成计数包含可能已经离盘但还没到 HBM 的量, 保守重复计作盘剩余工作可能影响 CIR。要提高在线预测确定性, 需要 SSU 合作报告命令服务/完成边界或可用的服务保证, 不能只增加 Python 公式。

7 文件和复现

主目录只保留最终 PDF, JSON/CSV 在 data, 图在 figures, Markdown 在 docs。没有修改 sim.py、continuous_batch_sim.py、continuous_prefill_client.py、policy_logic.py; 每组输出保存核心与策略源码哈希、原输入指纹、提交种子和原生 invariants。

实验期间控制器经历三版: 初始版、修正新事件的最小更新间隔、增加局部 stall 交换; 旧零间隔模式未改分配算法。不是所有输出都由当前源码版本生成。各历史版本按 SHA 归档于 data/source_versions, 最终可运行源文件与原始 data 打包在 data/final_source_bundle.tar.gz。审计列出每份输出对应版本并验证归档内容, 不用当前文件冒充历史版本。统计窗口也从逐层原始事件重新计算, 而不是只信缓存汇总数。

```
python -B -m pytest -q
python -B validate_multi_ssu_stall_predictor.py
python -B sweep_multi_ssu_experiments.py --phase primary --workers 6
python -B sweep_multi_ssu_experiments.py --phase holdout --workers 4
python -B sweep_multi_ssu_experiments.py --phase ablation --workers 4
python -B sweep_multi_ssu_experiments.py --phase hotspot --workers 4
python -B sweep_multi_ssu_experiments.py --phase interchange --workers 4
python -B build_multi_ssu_report.py
```

若只跑一组：

```
python -B run_multi_ssu_stall_experiments.py --num-ssu 7 \  
  --regime near --seed 20260906 --strategy deadline
```

增加 --assignment fluid 即 B。公式文件、CIR 控制器、在线选卡器分别为 multi_ssu_stall_predictor.py、multi_ssu_qos_controller.py、multi_ssu_npu_assignment.py。这是可复现实验适配器，不是生产驱动；下一步应根据本轮收益与成本决定是否值得接入真实控制链路，不能先承诺极大提升再挑结果。